Table des matières

Exercices corrigés 01 — Paysage SQL et modèle relationnel 1
Questions courtes 1
Questions pratiques 3
Diagnostics 4
Questions à développer 5
Mises en situation 5
Corrigé 7

Exercices corrigés 01 — Paysage SQL et modèle relationnel

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures et trois quarts pour y répondre, le corrigé non compris.

Les repères de terrain. Ce cours n'a qu'un laboratoire, sur l'entrepôt : les énoncés pratiques se traitent donc dans DuckDB, sur `adventureworksdw.duckdb` ouvert par `USE dbo;`, et portent la marque `[OLAP]`. Quelques questions parlent du monde transactionnel sans rien à exécuter : elles préparent le Cours 02, où le double chemin commence pour de bon.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les cinq mots. Rangez data, database, DBMS, RDBMS et SQL du plus concret au plus abstrait, et donnez une phrase par terme.
2. Le mot de trop. « DuckDB est un SQL. » Corrigez la phrase et dites quelle confusion elle trahit.
3. Le tableur. Donnez trois choses qu'un RDBMS garantit et qu'un tableur ne garantit pas.
4. Le R de RDBMS. À quoi renvoie-t-il exactement, et par quel mécanisme concret cette propriété se manifeste-t-elle dans une base ?
5. Les cinq familles. Nommez-les par leur sigle et donnez une commande représentative de chacune.

© 2026 iAlexMG. Tous droits réservés.

6. L’annulation. Le DML s’annule tant qu’on n’a pas validé. Qu’en est-il du DDL sur les deux moteurs de cette formation, et ailleurs ? Quelle conséquence pratique en tirez-vous avant de taper `DROP` ?
7. Le classement de `SELECT`. Certains comptent quatre familles au lieu de cinq. Quelle commande change de camp, et quelle version vaut-il mieux donner en entretien ?
8. `TRUNCATE` et `DELETE`. Les deux vident une table. Dans quelle famille se range chacun, et qu’est-ce que cette différence de famille change en pratique ? Dites aussi ce qu’elle ne change pas sur nos deux moteurs.
9. La norme. SQL est normalisé, et pourtant les moteurs diffèrent. Expliquez ce paradoxe apparent en une phrase, puis donnez un exemple vu dans ce cours.
10. OLTP et OLAP. Donnez la vocation de chacun en une phrase, et le nom du moteur de cette formation qui représente chaque camp. Situez ensuite les entrepôts de données dans le cloud par rapport à ces deux-là.
11. La clé primaire. Que garantit-elle, et qu’est-ce qu’une clé de substitution dans un entrepôt ?
12. La clé étrangère. Que garantit-elle, et quelle opération du langage cette garantie rend-elle possible ?
13. Le schéma. Dans `dbo.DimProduct`, que désigne `dbo`, et pourquoi ce préfixe existe-t-il ?
14. L’ordre des lignes. Une table a-t-elle un ordre ? Que se passe-t-il si vous omettez `ORDER BY` et que vous vous fiez à ce que vous voyez ?
15. L’étoile. Décrivez la forme d’un schéma en étoile en nommant ses deux sortes de tables, et dites ce que chacune contient.
16. L’ordre d’exécution. Donnez les sept étapes dans l’ordre où le moteur les exécute.

1. Le nom inconnu. Vous créez un alias dans le `SELECT` et vous l’utilisez dans le `WHERE`. PostgreSQL refuse, DuckDB accepte. Expliquez qui suit la norme et pourquoi, puis dites pourquoi le même alias fonctionne dans le `ORDER BY` des deux côtés.
2. `WHERE` ou `HAVING`. Donnez la règle de partage en une phrase, en la rattachant à l’ordre d’exécution plutôt qu’à une convention.
3. Les quatre jointures. Nommez-les et dites ce que chacune garde.
4. Le piège du `LEFT JOIN`. Décrivez-le, puis donnez la règle qui dit où placer une condition, dans le `ON` ou dans le `WHERE`.
5. `NULL`. Qu’est-ce que ce n’est pas, et que vaut une comparaison avec `NULL` ?
6. Les trois pièges de `NULL`. Nommez-les et donnez la parade de chacun.
7. Les totaux et les `NULL`. Que valent `COUNT(*)` et `COUNT(Color)` sur une table de 500 produits dont 250 ont une couleur à `NULL` ? Et `AVG(ListPrice)` si dix produits ont un prix à `NULL` ?

© 2026 iAlexMG. Tous droits réservés.

8. Le monde d'à côté. Sur la version transactionnelle d'AdventureWorks, les ventes ne vivent pas dans une table de faits. Où vivent-elles, et qu'est-ce que cela change au nombre de jointures d'une question analytique ?

9. `NULL` ou « NA ». Dans `DimProduct`, la colonne `Color` ne contient aucun `NULL`, mais vaut « NA » sur 254 lignes. Qu'est-ce qu'une valeur sentinelle, pourquoi un entrepôt en met-il dans ses dimensions, et que perd-on à le faire ?

10. `USING` ou `ON`. Qu'est-ce que `USING (ProductKey)` change dans le résultat par rapport à `ON p.ProductKey = f.ProductKey`, et dans quel cas ne faut-il pas l'employer ?

11. `<>` et `IS DISTINCT FROM`. Que valent `NULL <> 5`, `NULL IS DISTINCT FROM 5` et `NULL IS DISTINCT FROM NULL` ? Dites lequel des trois résultats le `WHERE` garde.

Questions pratiques

Écrivez la requête complète avant de regarder. Tapez-la vraiment dans la ligne de commande DuckDB, sur l'entrepôt ouvert par `USE dbo;` : le SQL s'apprend avec les doigts.

P1. `[OLAP]` Les produits noirs. Affichez le nom et le prix des produits de couleur `'Black'`, du plus cher au moins cher.

P2. `[OLAP]` Le nombre de produits par sous-catégorie. Pour chaque sous-catégorie, affichez son nom et le nombre de produits qu'elle contient.

P3. `[OLAP]` Les produits sans sous-catégorie. Listez les noms des produits dont la sous-catégorie n'est pas renseignée.

P4. `[OLAP]` Compter les vraies couleurs. En une seule requête, affichez le nombre total de lignes de `DimProduct` et le nombre de lignes qui ont vraiment une couleur.

P5. `[OLAP]` Le filtre et le total. Affichez les produits dont le prix dépasse 1 000 dollars, avec leur sous-catégorie en toutes lettres. Les produits sans sous-catégorie doivent apparaître quand même, avec une mention lisible plutôt qu'un `NULL`.

P6. `[OLAP]` La question analytique type. Affichez les dix produits les plus vendus en ligne, par quantité totale, avec leur nom lisible. Dites ensuite quelles tables vous avez dû joindre et pourquoi.

P7. `[OLAP]` Le filtre au bon étage. Affichez les sous-catégories qui comptent plus de quinze produits plus chers que la moyenne du catalogue, avec ce compte. Attention : il y a deux filtres, et ils ne vont pas au même endroit. Dites ensuite, en une phrase, ce que le premier filtre fait au compte du second.

P8. `[OLAP]` La preuve du piège. Écrivez deux requêtes qui comptent les lignes sans prix de liste, l'une avec `IS NULL`, l'autre avec `= NULL`. Exécutez les deux et notez les deux résultats. Refaites ensuite l'exercice sur `Color`, et expliquez pourquoi les deux zéros obtenus ne veulent pas dire la même chose.

P9. `[OLAP]` Le `LEFT JOIN` dégradé. Écrivez une requête qui liste tous les produits avec leur sous-catégorie, puis ajoutez une condition sur la sous-catégorie dans le `WHERE`. Comptez les lignes avant et après, expliquez l'écart, puis corrigez pour garder tous les produits.

© 2026 iAlexMG. Tous droits réservés.

P10. (avancé) La division protégée. Pour chaque produit, affichez le rapport entre son prix de liste et son coût standard (`StandardCost`). Rien ne garantit qu'un coût ne vaudra jamais zéro : traitez le cas proprement, dites ce qui se passerait sans cette précaution, puis vérifiez combien de coûts valent zéro et combien sont `NULL` dans cette base.

P11. (avancé) Le compte qui ment. Écrivez une requête qui joint `DimProduct` à `FactInternetSales` et qui compte les produits. Comparez le résultat à un simple `COUNT(*) FROM dbo.DimProduct`. Expliquez l'écart dans les deux sens, c'est-à-dire pourquoi le compte joint peut être à la fois plus grand et plus petit.

P12. (avancé) Trois façons de compter la même chose. Sur `DimProduct`, écrivez trois requêtes qui donnent le nombre de lignes ayant une vraie couleur, « NA » exclu, en utilisant trois mécanismes différents. Vérifiez qu'elles concordent, puis dites laquelle vous préférez et pourquoi.

P13. [OLAP] Le bilan des deux côtés. En une seule requête, rapprochez `DimProduct` et `DimProductSubcategory` par un `FULL OUTER JOIN`, et dites combien de produits n'ont pas de sous-catégorie et combien de sous-catégories n'ont aucun produit.

P14. [OLAP] Le prix qui n'est pas nul. Comptez les produits dont le prix de liste n'est pas 0, une fois avec `<>`, une fois avec `IS DISTINCT FROM`. Expliquez l'écart, et dites quel compte répond à la question posée.

Diagnostics

On vous montre du code qui ne fait pas ce que son auteur croit. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. Un débutant veut les produits dont le prix dépasse le double du coût :

```sql
SELECT EnglishProductName, ListPrice, StandardCost,
       ListPrice / StandardCost AS ratio
FROM dbo.DimProduct
WHERE ratio > 2;
```

Sur DuckDB, elle passe. Recopiée dans PostgreSQL sur une table de même forme, elle échoue : le moteur répond que la colonne `ratio` n'existe pas. Le débutant conclut que PostgreSQL est capricieux. Expliquez la vraie cause, dites pourquoi le message est trompeur, donnez les deux corrections possibles, et signalez un second défaut de cette requête qui n'a rien à voir avec le premier. Ce second défaut ne lève aucune erreur sur l'entrepôt : dites ce qu'il y produit à la place.

E2. Une analyste veut les sous-catégories qui vendent pour plus de 100 000 dollars :

```sql
SELECT s.EnglishProductSubcategoryName, SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimProduct            AS p ON p.ProductKey = f.ProductKey
JOIN dbo.DimProductSubcategory AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
WHERE SUM(f.SalesAmount) > 100000
GROUP BY s.EnglishProductSubcategoryName;
```

Le moteur refuse. Expliquez pourquoi en vous appuyant sur l'ordre d'exécution, corrigez, puis dites ce qui changerait si elle voulait au contraire ne compter que les ventes postérieures à 2012.

© 2026 iAlexMG. Tous droits réservés.

E3. Un développeur veut savoir si des sous-catégories ne sont rattachées à aucun produit :

```sql
SELECT EnglishProductSubcategoryName
FROM dbo.DimProductSubcategory
WHERE ProductSubcategoryKey NOT IN (SELECT ProductSubcategoryKey FROM dbo.DimProduct);
```

Le résultat est vide, et il en conclut que toutes les sous-catégories sont utilisées. A-t-il le droit de conclure ça ? Expliquez le mécanisme exact, proposez une manipulation en deux lignes qui prouve que la requête est incapable de répondre quoi qu'il arrive, puis donnez deux écritures correctes.

E4. Une analyste veut savoir quelle part du catalogue n'a pas de couleur renseignée :

```sql
SELECT COUNT(*) FROM dbo.DimProduct WHERE Color IS NULL;
```

Elle obtient zéro, et écrit dans son rapport que toutes les couleurs sont renseignées. Dites ce qui cloche, par quelle requête elle aurait pu s'en apercevoir, et corrigez. Dites ensuite pourquoi le chiffre corrigé mérite lui-même une nuance avant d'entrer dans le rapport.

E5. Un analyste fait le tour des tailles du catalogue :

```sql
SELECT COUNT(*) FROM dbo.DimProduct WHERE Size = 'M';
SELECT COUNT(*) FROM dbo.DimProduct WHERE Size <> 'M';
```

Il obtient 14 et 285, et s'étonne : 14 et 285 ne font pas les 606 lignes de la table. Où sont passés les autres produits, et comment écrire le second compte pour que la somme tombe juste ?

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. L'ordre d'exécution comme outil de diagnostic. On présente souvent l'ordre d'exécution comme une liste à mémoriser. Défendez l'idée que c'est en réalité un outil de raisonnement, et montrez-le : prenez trois erreurs de débutant qui semblent sans rapport entre elles, et expliquez comment la même question, « à cette étape, cette colonne existe-t-elle », les résout toutes les trois. Terminez en disant ce que cette grille ne permet pas d'expliquer.

B. Pourquoi deux modèles pour la même entreprise. AdventureWorks existe en version transactionnelle normalisée et en version entrepôt dénormalisée. Expliquez à quelqu'un qui trouve ça redondant pourquoi les deux existent, ce que chacune optimise, et ce qu'elle sacrifie pour y arriver. Prenez un exemple concret d'opération qui est facile d'un côté et pénible de l'autre, dans les deux sens.

C. `NULL`, une bonne ou une mauvaise idée. La valeur `NULL` cause une part notable des bogues de SQL, au point que certains auteurs recommandent de l'interdire dans les modèles. Présentez honnêtement les deux positions : ce que `NULL` apporte que rien d'autre n'apporte, et ce qu'il coûte en pièges. Concluez par la règle que vous adopteriez en concevant une table.

Mises en situation

© 2026 iAlexMG. Tous droits réservés.

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Librairie Rive-Gauche, l'inventaire qui ne balance pas. La librairie tient son stock dans une petite base PostgreSQL. Chaque soir, un employé lance une requête qui compte les livres en stock par rayon, et le total ne correspond jamais tout à fait à l'inventaire physique. En regardant la table, vous voyez que la colonne `rayon` est parfois vide, que certains livres ont une quantité à `NULL` plutôt qu'à zéro, et que la requête utilise un `JOIN` vers la table des rayons.

Expliquez, sans écrire une ligne de SQL pour l'instant, les trois mécanismes distincts par lesquels ce rapport peut perdre des livres. Puis écrivez la requête corrigée, et dites pour chaque correction lequel des trois mécanismes elle règle. Terminez par la question de fond : faut-il corriger la requête, ou faut-il corriger les données, et qu'est-ce qui décide ?

M2. Commission scolaire des Deux-Rives, la question de vocabulaire. Vous arrivez dans une équipe qui utilise indifféremment les mots « base de données », « serveur », « SQL » et « Oracle » pour désigner la même chose. On vous demande de préparer une page pour les nouveaux arrivants.

Rédigez le raisonnement que vous suivriez : quelles distinctions sont indispensables, lesquelles sont du pinaillage, et comment vous vous y prenez pour qu'une clarification de vocabulaire ne passe pas pour une leçon. Justifiez notamment pourquoi la confusion entre le langage et le moteur a des conséquences concrètes, avec un exemple où elle mène à une mauvaise décision technique.

M3. `(avancé)` Distribution alimentaire Beauharnois, la mauvaise table. L'entreprise a un entrepôt de données alimenté chaque nuit et une base transactionnelle de production. Un analyste vient de brancher son tableau de bord directement sur la base de production, parce que, dit-il, « les données sont plus fraîches et le résultat est le même ». Son tableau tourne toutes les cinq minutes et interroge trois ans d'historique de commandes.

Répondez-lui. Vous devez traiter quatre choses : ce qui est juste dans son raisonnement, ce qui est faux, ce que sa décision coûte à des gens qui ne sont pas dans la conversation, et à quelles conditions précises il aurait raison. Terminez en formulant la question qu'il aurait dû se poser en premier, et dites pourquoi c'est celle-là et pas la question de la fraîcheur.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Du plus concret au plus abstrait : data, database, DBMS, RDBMS, SQL. La *data*, ce sont les faits bruts, un prix ou un nom. La *database* est un ensemble de données rangées pour être retrouvées vite. Le *DBMS* est le logiciel qui gère cette base. Le *RDBMS* est un DBMS qui range tout en tables reliées par des clés. `SQL` est le langage qu'on parle au RDBMS.

Théorie : `Theorie_01_SQL.md`, section 1.

2. « DuckDB est un RDBMS », ou « DuckDB comprend le SQL ». La phrase confond le langage et le moteur, ce qui est la confusion la plus répandue du domaine, et la plus coûteuse : elle laisse croire qu'apprendre un moteur revient à apprendre le langage, et inversement qu'un changement de moteur oblige à tout réapprendre.

Théorie : `Theorie_01_SQL.md`, section 1.

3. Des clés qui interdisent les doublons d'identifiant, des liens garantis entre tables (une vente ne peut pas désigner un produit inexistant), et l'écriture simultanée par plusieurs personnes sans que le fichier se corrompe ou se verrouille. On peut ajouter les types de données imposés et les transactions. Ces garanties sont celles d'un moteur transactionnel comme PostgreSQL ; notre entrepôt, lui, ne déclare aucune clé, et remplace donc la garantie par un contrôle.

Théorie : `Theorie_01_SQL.md`, section 4.

4. Le « R » renvoie à relationnel, c'est-à-dire au fait que les tables sont reliées entre elles. Le mécanisme concret est la paire clé primaire et clé étrangère : une colonne d'une table pointe vers l'identifiant d'une autre. Sur PostgreSQL, le moteur fait respecter ce lien ; sur l'entrepôt de cette formation, il n'y a aucune contrainte déclarée, et le lien n'est qu'une convention que les chargements doivent honorer.

Théorie : `Theorie_01_SQL.md`, sections 1 et 4.

5. `DDL` avec `CREATE TABLE`, `DML` avec `INSERT`, `DQL` avec `SELECT`, `DCL` avec `GRANT`, `TCL` avec `COMMIT`.

Théorie : `Theorie_01_SQL.md`, section 2.

6. Le DML s'annule, tant qu'on n'a pas fait `COMMIT` : un `ROLLBACK` remet tout comme avant. Pour le DDL, la réponse dépend du moteur, et les deux nôtres sont du même côté : sur DuckDB comme sur PostgreSQL, un `CREATE TABLE`, un `TRUNCATE` ou un `DROP TABLE` lancés entre un `BEGIN` et un `ROLLBACK` sont défaits, table et contenu revenus comme avant, ce que montre le Cours 03. Ailleurs, le DDL est validé d'office, en auto-commit, donc irrécupérable : c'est le cas de Snowflake, d'Oracle et de MySQL, d'après leur documentation. La conséquence pratique ne change pas pour autant, et c'est le point de la question : avant un `DROP`, on ne compte pas sur le filet, parce qu'il dépend d'un moteur qu'on ne choisit pas toujours et d'une transaction qu'on a peut-

© 2026 iAlexMG. Tous droits réservés.

être oublié d'ouvrir. On vérifie sur quelle base et quel schéma on est connecté, et on sauvegarde s'il y a le moindre doute. La prudence doit venir avant la commande, pas après.

Théorie : `Theorie_01_SQL.md`, section 2.

7. Le `SELECT`, que certains rangent dans le DML puisqu'il manipule des données. La version à cinq familles, avec le DQL séparé, est la plus répandue et la plus claire : c'est celle qu'on donne en entretien, quitte à mentionner qu'on connaît l'autre découpage.

Théorie : `Theorie_01_SQL.md`, section 2.

8. `DELETE` est du DML, `TRUNCATE` est du DDL. La différence de famille n'est pas un détail de classement : `DELETE` journalise chaque ligne et peut porter un `WHERE`, `TRUNCATE` vide la table d'un coup, sans condition possible. `TRUNCATE` est donc beaucoup plus rapide et beaucoup plus brutal, ce qui est le même fait vu de deux côtés.

Ce que la différence de famille ne change pas sur nos deux moteurs, c'est la réversibilité. Sur beaucoup de moteurs, `TRUNCATE` est validé d'office et ne se reprend pas, et c'est l'argument qu'on entend le plus souvent pour le distinguer de `DELETE` ; sur DuckDB comme sur PostgreSQL, un `TRUNCATE` lancé entre un `BEGIN` et un `ROLLBACK` est annulé et les lignes reviennent. Retenez donc la vraie différence, la vitesse et l'absence de `WHERE`, plutôt que la formule toute faite sur le retour en arrière, qui n'est vraie que sur certains moteurs.

Théorie : `Theorie_01_SQL.md`, section 2.

9. La norme fixe le cœur du langage, et laisse chaque éditeur libre d'ajouter des extensions. Les bases sont donc identiques, les raffinements diffèrent. L'exemple du cours est `QUALIFY`, qui tourne sur DuckDB et manque à PostgreSQL. L'alias du `SELECT` repris dans le `WHERE` en est un second, et il est plus retors : ce n'est pas une extension visible, c'est une tolérance silencieuse, qui fait passer sur un moteur une requête que le voisin refuse.

Théorie : `Theorie_01_SQL.md`, section 3.

10. L'OLTP fait tourner l'entreprise au quotidien : beaucoup de petites écritures et lectures, en continu, sur peu de lignes à la fois, et c'est PostgreSQL ici. L'OLAP analyse l'entreprise : peu de requêtes, mais qui balaient énormément de lignes, et c'est DuckDB.

Les entrepôts de données dans le cloud, comme Snowflake, BigQuery ou Redshift, sont du côté OLAP : même métier que DuckDB, même genre de requêtes, mais sur des machines louées à l'heure plutôt que sur la vôtre. Ce qui les sépare de DuckDB n'est donc pas la vocation, c'est l'échelle et le modèle de coût. Le langage, lui, est le même à peu de chose près, et c'est pourquoi ce qu'on apprend ici se transporte.

Théorie : `Theorie_01_SQL.md`, section 3.

11. Quand elle est déclarée, elle garantit qu'aucune ligne n'a la même valeur qu'une autre sur cette colonne, et qu'elle n'est jamais nulle : chaque ligne est donc désignable sans ambiguïté. Dans notre entrepôt, aucune clé n'est déclarée, et `ProductKey` n'est donc une clé que par convention : c'est un `COUNT(*)` contre un `COUNT(DISTINCT …)` qui le vérifie. Une clé de substitution est un identifiant que l'entrepôt attribue lui-même, indépendant des codes du système source, ce qui lui permet de survivre à un changement de codification chez la source et d'accueillir plusieurs versions d'une même entité.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_01_SQL.md`, section 4.

12. Sur un moteur qui la fait respecter, elle garantit qu'une valeur qui pointe vers une autre table désigne une ligne qui existe vraiment. C'est ce lien qui rend les jointures fiables : sans lui, on peut toujours joindre, mais rien n'assure que chaque vente corresponde à un produit connu. C'est exactement la situation de l'entrepôt, sans contrainte : la jointure marche, et une vente orpheline disparaîtrait en silence d'une jointure interne. D'où le contrôle, plutôt que la confiance.

Théorie : `Theorie_01_SQL.md`, section 4.

13. `dbo` est le schéma, c'est-à-dire le tiroir dans lequel la table est rangée. Le préfixe existe pour regrouper les tables qui vont ensemble et pour permettre à deux tables du même nom de coexister dans deux schémas différents. L'écriture complète est schéma point table.

Théorie : `Theorie_01_SQL.md`, section 5.

14. Non, une table n'a pas d'ordre. Sans `ORDER BY`, le moteur renvoie les lignes dans l'ordre qui l'arrange, et cet ordre peut changer d'une exécution à l'autre, notamment quand le volume grandit ou que le plan d'exécution change. Se fier à ce qu'on voit, c'est écrire un programme qui marche aujourd'hui et qui cassera sans qu'on ait rien modifié.

Théorie : `Theorie_01_SQL.md`, section 4.

15. Au centre, une table de faits : une ligne par événement mesurable, une vente par exemple, avec des mesures numériques et des clés. Autour, des tables de dimensions qui décrivent le contexte de cet événement, le produit, la date, le client, avec des libellés lisibles.

Théorie : `Theorie_01_SQL.md`, section 5.

16. `FROM` et les `JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`.

Théorie : `Theorie_01_SQL.md`, section 6.

17. Le `WHERE` s'exécute à l'étape 2, le `SELECT` à l'étape 5 : au moment où le `WHERE` est évalué, l'alias n'a pas encore été créé, donc il n'existe pas. C'est PostgreSQL qui suit la norme, et tout moteur qui la suit à la lettre refuse la même requête pour la même raison. DuckDB fait exception : il résout l'alias du `SELECT` avant de filtrer, et accepte la requête lui, s'exécute à l'étape 6, après le `SELECT` : la colonne est déjà calculée et porte déjà son nom, et il fonctionne donc des deux côtés.

La leçon à tirer n'est pas que DuckDB serait plus permissif et qu'on peut s'y fier. C'est qu'une requête écrite sur l'entrepôt peut casser telle quelle sur le transactionnel, sans qu'on ait rien fait de faux au sens du moteur qui l'a acceptée. Le double chemin de cette formation existe en bonne partie pour vous faire rencontrer ce genre d'écart pendant qu'il ne coûte rien.

Théorie : `Theorie_01_SQL.md`, section 6.

18. Le `WHERE` filtre des lignes avant tout regroupement, le `HAVING` filtre des groupes après. Ce n'est pas une convention, c'est une conséquence de l'ordre : à l'étape du `WHERE`, aucun agrégat n'existe encore, donc aucune condition portant sur un total ne peut y être évaluée.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_01_SQL.md`, section 6.

19. `INNER JOIN` garde les lignes qui correspondent des deux côtés. `LEFT JOIN` garde chaque ligne de la table de gauche, celle écrite avant le `JOIN`, et met des `NULL` dans les colonnes de la table de droite quand celle-ci n'a rien qui corresponde. `FULL OUTER JOIN` garde chaque ligne des deux tables, et met des `NULL` du côté qui n'a rien à associer. `SELF JOIN` joint une table à elle-même, pour comparer deux lignes du même ensemble.

Théorie : `Theorie_01_SQL.md`, section 7.

20. Le `LEFT JOIN` met des `NULL` dans les colonnes de la table de droite pour les lignes de la table de gauche restées sans correspondance ; une condition placée dans le `WHERE` sur une colonne de la table de droite rejette ces `NULL`, et la jointure se comporte alors comme une jointure interne. La règle : une condition qui décrit comment relier les tables va dans le `ON`, une condition qui filtre le résultat final va dans le `WHERE`.

Théorie : `Theorie_01_SQL.md`, sections 7 et 8.

21. Ce n'est ni zéro, ni une chaîne vide : c'est une valeur inconnue. Une comparaison avec `NULL` ne vaut ni vrai ni faux mais inconnu, et le `WHERE` ne garde que ce qui est franchement vrai.

Théorie : `Theorie_01_SQL.md`, section 8.

22. Premièrement, `= NULL` ne renvoie jamais rien, et la parade est `IS NULL`. Deuxièmement, les agrégats ignorent les `NULL`, ce qui est souvent souhaitable mais qu'il faut savoir, et la parade est de choisir sciemment entre `COUNT(*)` et `COUNT(colonne)`. Troisièmement, `NOT IN` devient stérile si la liste contient un seul `NULL`, et la parade est `NOT EXISTS`.

Théorie : `Theorie_01_SQL.md`, section 8.

23. `COUNT(*)` vaut 500, `COUNT(Color)` vaut 250. `AVG(ListPrice)` calcule la moyenne des 490 produits qui ont un prix : les dix autres ne sont ni comptés au numérateur ni au dénominateur. C'est généralement ce qu'on veut, mais il faut le savoir, car la moyenne obtenue n'est pas la moyenne des

Théorie : `Theorie_01_SQL.md`, section 8.

24. Elles vivent dans deux tables, un en-tête de commande et ses lignes de commande, reliées par l'identifiant de commande. Une question analytique demande donc au moins une jointure de plus, et souvent davantage, parce que le contexte de la vente (le produit, le territoire, le pays) est lui-même éclaté dans d'autres tables. Le calcul est le même, le chemin pour y arriver est plus long.

Théorie : `Theorie_01_SQL.md`, section 3.

25. Une valeur sentinelle est une vraie valeur, choisie par convention pour dire « rien » : ici la chaîne « NA ». Un entrepôt en met dans ses dimensions parce qu'un libellé se lit, se filtre et se regroupe comme n'importe quelle catégorie, là où un `NULL` laisserait une case vide dans un rapport. Ce qu'on perd, c'est que le moteur ne sait plus qu'il s'agit d'une absence : `IS NULL` ne la trouve pas, `COUNT(Color)` la compte, et « NA » mélange sans le dire des cas différents. Quand on a besoin de retrouver l'absence, `NULLIF(Color, 'NA')` reconvertit le libellé en `NULL`.

Théorie : `Theorie_01_SQL.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.

26. Les deux relient les mêmes lignes. Avec `ON`, la clé existe deux fois dans le résultat, une par table, et il faut la préfixer pour la nommer ; avec `USING`, elle n'existe qu'une fois et se nomme sans préfixe. On ne l'emploie pas quand les deux colonnes portent le même nom sans désigner la même chose : `USING` joint sur les noms, pas sur le sens, et deux `ModifiedDate` se joindraient sans un mot.

Théorie : `Theorie_01_SQL.md`, section 7.

27. `NULL <> 5` vaut inconnu (`NULL`), `NULL IS DISTINCT FROM 5` vaut vrai, et `NULL IS DISTINCT FROM NULL` vaut faux. Le `WHERE` ne garde que le deuxième, le seul franchement vrai : c'est pourquoi une condition `<>` perd en silence les lignes où la colonne vaut `NULL`.

Théorie : `Theorie_01_SQL.md`, section 8.

Questions pratiques

P1. `[OLAP]`

```sql
SELECT EnglishProductName, ListPrice
FROM dbo.DimProduct
WHERE Color = 'Black'
ORDER BY ListPrice DESC;
```

Théorie : `Theorie_01_SQL.md`, section 2.

P2. `[OLAP]`

```sql
SELECT s.EnglishProductSubcategoryName AS sous_categorie,
       COUNT(DISTINCT p.ProductAlternateKey) AS nb_produits
FROM dbo.DimProductSubcategory AS s
JOIN dbo.DimProduct AS p ON p.ProductSubcategoryKey = s.ProductSubcategoryKey
GROUP BY s.EnglishProductSubcategoryName
ORDER BY nb_produits DESC;
```

Le `DISTINCT` n'est pas une précaution de style, il est la réponse à la question posée. La question demandait des produits, et une ligne de `DimProduct` n'est pas un produit mais une version de produit : le Labo 00 l'a montré, la dimension historise, et le même code source y revient à chaque changement. Écrire `COUNT(*)` compte donc les versions. L'écart touche 9 des 37 sous-catégories, et il n'est pas décoratif : il change la tête du classement. En lignes, Road Frames arrive premier avec 70 ; en produits, il tombe à 33 et passe derrière Road Bikes, qui en compte 43. Un rapport construit sur `COUNT(*)` aurait désigné la mauvaise sous-catégorie, sans jamais lever d'erreur.

Théorie : `Theorie_01_SQL.md`, sections 6 et 7.

P3. `[OLAP]`

```sql
SELECT EnglishProductName
FROM dbo.DimProduct
WHERE ProductSubcategoryKey IS NULL;
```


© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_01_SQL.md`, section 8.

P4. [OLAP]

```sql
SELECT COUNT(*)                    AS nb_lignes,
       COUNT(Color)                AS nb_avec_couleur_ecrite,
       COUNT(NULLIF(Color, 'NA')) AS nb_vraiment_colorees
FROM dbo.DimProduct;
```

Réponse : 606, 606 et 352. La deuxième colonne est la réponse naïve, et elle est fausse : `Color` ne contient aucun `NULL`, les produits sans couleur y valent « NA », et `COUNT` les compte comme les autres. `NULLIF(Color, 'NA')` rend `NULL` à ces 254 lignes, et `COUNT` peut enfin les écarter. C'est la section 9 de `Theorie_01_SQL.md`.

Théorie : `Theorie_01_SQL.md`, sections 8 et 9.

P5. [OLAP]

```sql
SELECT p.EnglishProductName AS produit,
       p.ListPrice,
       COALESCE(s.EnglishProductSubcategoryName, 'Sans sous-catégorie') AS sous_categorie
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s
       ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
WHERE p.ListPrice > 1000
ORDER BY p.ListPrice DESC;
```

Le `LEFT JOIN` garde les produits sans sous-catégorie, et le `COALESCE` remplace le `NULL` qui en résulte. Le filtre sur le prix, lui, porte sur la table de gauche, `DimProduct` : il peut donc rester dans le `WHERE` sans dégrader la jointure. C'est exactement la nuance de la question 20.

Théorie : `Theorie_01_SQL.md`, section 6.

P6. [OLAP]

```sql
SELECT p.EnglishProductName, SUM(f.OrderQuantity) AS total_vendu
FROM dbo.FactInternetSales AS f
JOIN dbo.DimProduct        AS p ON p.ProductKey = f.ProductKey
GROUP BY p.EnglishProductName
ORDER BY total_vendu DESC
LIMIT 10;
```

Deux tables. La table de faits porte la quantité, mais ne connaît le produit que par une clé numérique ; la dimension porte le nom lisible, mais ne sait rien des ventes. Il faut donc les deux, et c'est le geste fondamental de l'entrepôt : la mesure vient du fait, le libellé vient de la dimension.

Théorie : `Theorie_01_SQL.md`, sections 5 et 7.

© 2026 iAlexMG. Tous droits réservés.

P7. [OLAP]

```sql
SELECT s.EnglishProductSubcategoryName AS sous_categorie,
       COUNT(DISTINCT p.ProductAlternateKey) AS nb_produits_chers
FROM dbo.DimProduct AS p
JOIN dbo.DimProductSubcategory AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
WHERE p.ListPrice > (SELECT AVG(ListPrice) FROM dbo.DimProduct WHERE ListPrice > 0)
GROUP BY s.EnglishProductSubcategoryName
HAVING COUNT(DISTINCT p.ProductAlternateKey) > 15
ORDER BY nb_produits_chers DESC;
```

Attendu : deux sous-catégories, Road Bikes avec 39 produits et Mountain Bikes avec 22. Le `DISTINCT` est celui de P2, pour la même raison, et il pèse ici plus lourd encore : compté en lignes, ce classement aurait rendu quatre sous-catégories, Road Bikes à 49, Mountain Bikes à 28, Road Frames à 23 et Mountain Frames à 22. Les deux sous-catégories de cadres n'y entraient que par leurs versions. Le seuil de quinze ne veut rien dire tant qu'on n'a pas dit quinze de quoi.

Les deux filtres ne vont pas au même endroit, et l'ordre d'exécution dit pourquoi. Le prix est une propriété de la ligne, donc il se filtre dans le `WHERE`, avant tout regroupement. Le nombre de produits est une propriété du groupe, donc il se filtre dans le `HAVING`, après. Intervertir les deux ne donne pas un résultat différent, ça donne une requête refusée.

Reste la phrase demandée, et c'est elle qui compte. Le `WHERE` s'exécutant avant le regroupement, le `COUNT(*)` du `HAVING` ne compte pas les produits de la sous-catégorie : il compte ceux qui ont survécu au premier filtre, c'est-à-dire les produits chers. « Plus de quinze produits » et « plus de quinze produits chers » sont deux questions différentes, et cette requête répond à la seconde. Si vous vouliez la première, il faudrait compter avant de filtrer, donc une sous-requête ou une fenêtre, que les Cours 04 et 07 vous donneront. Retenez surtout le réflexe : devant un `HAVING COUNT(*)`, demandez-vous toujours ce que le `WHERE` a déjà retiré du paquet.

Théorie : `Theorie_01_SQL.md`, section 6.

P8. [OLAP]

```sql
SELECT COUNT(*) FROM dbo.DimProduct WHERE ListPrice IS NULL;   -- 211, le vrai compte
SELECT COUNT(*) FROM dbo.DimProduct WHERE ListPrice = NULL;    -- toujours zero
```

La seconde renvoie zéro, quelle que soit la base et quelles que soient les données. Elle ne teste rien : `ListPrice = NULL` s'évalue à inconnu pour chaque ligne, et le `WHERE` ne garde que le vrai. Ce qui rend ce piège redoutable, c'est que zéro est une réponse plausible.

Sur `Color`, les deux requêtes répondent zéro, mais les deux zéros ne disent pas la même chose. Celui de `= NULL` n'est pas une réponse, pour la raison qu'on vient de voir. Celui de `IS NULL` en est une, et elle est exacte : la colonne ne contient aucun `NULL`. Elle est seulement trompeuse, parce que les produits sans couleur y sont écrits « NA » (section 9 de `Theorie_01_SQL.md`).

Théorie : `Theorie_01_SQL.md`, section 8.

P9. [OLAP]

© 2026 iAlexMG. Tous droits réservés.

```sql
-- 1. tous les produits
SELECT COUNT(*)
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s
       ON s.ProductSubcategoryKey = p.ProductSubcategoryKey;

-- 2. la meme requete, avec une condition sur la table de DROITE dans le WHERE
SELECT COUNT(*)
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s
       ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
WHERE s.EnglishProductSubcategoryName IS NOT NULL;
```

L'écart correspond exactement aux produits sans sous-catégorie. Le `LEFT JOIN` les avait bien gardés, avec des `NULL` dans les colonnes de la sous-catégorie, puis le `WHERE` les a rejetés, puisqu'un `NULL` ne satisfait aucune condition. La jointure externe s'est donc comportée comme une jointure interne, et rien ne l'a signalé.

```sql
-- Correction : la condition descend dans le ON, chaque produit (table de gauche) reste
SELECT p.EnglishProductName, s.EnglishProductSubcategoryName
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s
       ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
      AND s.EnglishProductSubcategoryName LIKE 'M%';
```

Théorie : `Theorie_01_SQL.md`, section 7.

P10. (avancé)

```sql
SELECT EnglishProductName, ListPrice, StandardCost,
       ListPrice / NULLIF(StandardCost, 0) AS marge_relative
FROM dbo.DimProduct
ORDER BY marge_relative DESC NULLS LAST;
```

`NULLIF(StandardCost, 0)` transforme le zéro en `NULL`, et une division par `NULL` donne `NULL`. Le `NULLS LAST` du tri renvoie en fin de liste les produits sans marge calculable (Cours 02, section 1) ; il est ici explicite pour dire l'intention, mais DuckDB range déjà les `NULL` en dernier de lui-même, son réglage `default_null_order` valant `NULLS_LAST`.

Sans la précaution du `NULLIF`, le comportement dépend du moteur, et nos deux terrains illustrent justement les deux cas. `SELECT 10 / 0` lève une erreur sur PostgreSQL, « division par zéro », et la requête entière s'arrête. Sur DuckDB, la même expression rend `inf`, l'infini, sans avertissement : la requête réussit, et la valeur se propage. Une somme ou une moyenne qui rencontre un seul `inf` vaut `inf` à son tour. Le premier cas fait perdre une minute, le second fait perdre une semaine, parce qu'il produit un résultat qu'on peut recopier dans un rapport. La contamination se montre en trois lignes, sur l'entrepôt :

```sql
-- Une division par zero, sans NULLIF : DuckDB rend inf, sans erreur
SELECT 10 / 0 AS division_par_zero;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Et ce que cet inf fait a une somme et a une moyenne : les deux valent inf
SELECT SUM(x) AS somme, AVG(x) AS moyenne
FROM (SELECT UNNEST([1.0, 2.0, CAST(1.0 AS DOUBLE) / 0]) AS x);

SELECT COUNT(*) AS cout_null FROM dbo.DimProduct WHERE StandardCost IS NULL;   -- 211
SELECT COUNT(*) AS cout_zero FROM dbo.DimProduct WHERE StandardCost = 0;       -- 0
```

Dans cette base, aucun coût ne vaut zéro : les 211 lignes sans coût ont un `StandardCost` à `NULL`, et la division donne alors `NULL` d'elle-même. La précaution vise les données de demain, pas celles d'aujourd'hui. Une requête doit rester juste quand les données changent, et un coût à zéro saisi par erreur la semaine prochaine ne préviendra personne, surtout sur un moteur qui répond `inf` plutôt que de s'arrêter.

Théorie : `Theorie_01_SQL.md`, section 8.

P11. (avancé)

```sql
SELECT COUNT(*) FROM dbo.DimProduct;                          -- le catalogue entier

SELECT COUNT(*) AS lignes_jointes,
       COUNT(DISTINCT p.ProductKey) AS produits_vendus
FROM dbo.DimProduct        AS p
JOIN dbo.FactInternetSales AS f ON f.ProductKey = p.ProductKey;
```

Le compte joint est à la fois plus grand et plus petit, selon ce qu'on compte, et c'est le cœur de l'exercice. Il est beaucoup plus grand si l'on écrit `COUNT(*)` : chaque produit apparaît autant de fois qu'il a été vendu, ce qu'on appelle l'éclatement par jointure, et c'est la cause la plus fréquente de totaux gonflés en analytique. Il est plus petit si l'on écrit `COUNT(DISTINCT p.ProductKey)` : les produits jamais vendus en ligne disparaissent, puisque la jointure interne les écarte.

La leçon tient en une phrase : après une jointure, `COUNT(*)` ne compte plus des produits, il compte des lignes, et personne ne vous préviendra du changement de grain.

Théorie : `Theorie_01_SQL.md`, sections 7 et 8.

P12. (avancé)

```sql
SELECT COUNT(NULLIF(Color, 'NA'))                    AS methode_1 FROM dbo.DimProduct;
SELECT COUNT(*)                                      AS methode_2 FROM dbo.DimProduct WHERE Color <> 'NA';
SELECT SUM(CASE WHEN Color <> 'NA' THEN 1 ELSE 0 END) AS methode_3 FROM dbo.DimProduct;
```

Les trois donnent 352, et elles écarteraient aussi un vrai `NULL` s'il en apparaissait un jour : une comparaison avec `NULL` vaut inconnu, donc ni le `WHERE` ni le `CASE` ne la retiennent. Je préfère la première, parce qu'elle nomme la convention (« NA » veut dire absent) et rend ensuite la main aux règles ordinaires des `NULL`. La deuxième est la plus explicite, donc la meilleure quand la requête sera lue par quelqu'un qui débute. La troisième, un `CASE` (Cours 02, section 3) glissé dans une somme, n'a d'intérêt que si l'on doit compter plusieurs conditions différentes côte à côte, et c'est précisément le motif du Cours 06.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_01_SQL.md`, section 8.

P13. `[OLAP]`

```sql
SELECT COUNT(*)                      AS lignes,
       COUNT(p.ProductKey)           AS avec_produit,
       COUNT(s.ProductSubcategoryKey) AS avec_sous_categorie
FROM dbo.DimProduct AS p
FULL OUTER JOIN dbo.DimProductSubcategory AS s
            ON s.ProductSubcategoryKey = p.ProductSubcategoryKey;
```

Résultat : 606 lignes, 606 avec un produit, 397 avec une sous-catégorie. Les 209 lignes sans sous-catégorie (606 − 397) sont des produits qui n'en ont pas, les pièces détachées pour l'essentiel. Aucune ligne n'est sans produit, donc les 37 sous-catégories ont toutes au moins un produit. Un `LEFT JOIN` parti des produits aurait donné le premier chiffre, pas le second.

Théorie : `Theorie_01_SQL.md`, sections 7 et 8.

P14. `[OLAP]`

```sql
SELECT COUNT(*) AS avec_diff     FROM dbo.DimProduct WHERE ListPrice <> 0;               -- 395
SELECT COUNT(*) AS avec_distinct FROM dbo.DimProduct WHERE ListPrice IS DISTINCT FROM 0; -- 606
```

L'écart, 211 lignes, ce sont les produits sans prix : `NULL <> 0` vaut inconnu, et `<>` les écarte. Aucun produit de l'entrepôt n'a un prix de 0, donc les deux comptes répondent à deux questions. « Combien de produits ont un vrai prix ? », c'est 395, et `<>` y répond par chance : la forme qui le dit franchement est `ListPrice IS NOT NULL AND ListPrice <> 0`. « Combien de produits ne sont pas à 0 ? », c'est 606, et seul `IS DISTINCT FROM` le dit. Écrire l'intention plutôt que compter sur le comportement de `NULL` est la règle.

Théorie : `Theorie_01_SQL.md`, section 8.

Diagnostics

E1. Le `WHERE` s'exécute avant le `SELECT`, donc l'alias `ratio` n'existe pas encore au moment où la condition est évaluée. Le message est trompeur parce qu'il parle d'une colonne inconnue, ce qui suggère une faute de frappe ou une table mal choisie, alors que le nom est parfaitement correct et le problème purement temporel. Ce n'est pas un caprice de PostgreSQL, c'est la règle de la norme, et tout moteur qui la suit à la lettre refuse la même requête de la même façon. DuckDB est l'exception, qui résout l'alias du `SELECT` avant de filtrer (Cours 02, section 1), et c'est précisément ce qui a mis le débutant sur une fausse piste : la requête ayant marché une fois, il a cherché le coupable du côté du moteur qui refuse, et non du côté de celui qui tolère.

```sql
-- Correction 1 : repeter l'expression
SELECT EnglishProductName, ListPrice, StandardCost,
       ListPrice / NULLIF(StandardCost, 0) AS ratio
FROM dbo.DimProduct
WHERE ListPrice / NULLIF(StandardCost, 0) > 2;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Correction 2 : envelopper, et filtrer a l'exterieur
SELECT * FROM (
    SELECT EnglishProductName, ListPrice, StandardCost,
           ListPrice / NULLIF(StandardCost, 0) AS ratio
    FROM dbo.DimProduct
) AS t
WHERE ratio > 2;
```

Le second défaut est la division par `StandardCost` sans protection. Aucun coût ne vaut zéro aujourd'hui dans cette base (les coûts absents sont `NULL`), mais rien ne l'interdit, et une requête doit survivre aux données de demain. C'est la question P10, et il fallait la voir ici sans qu'on la signale.

Ce que ce second défaut produirait sur l'entrepôt mérite d'être dit, parce que ce n'est pas une erreur : DuckDB ne s'arrête pas sur une division par zéro, il rend `inf`. La ligne fautive ressortirait donc en tête du classement, avec une marge infinie, et elle empoisonnerait tout total calculé sur cette colonne. Les deux défauts de cette requête sont ainsi de natures opposées, et c'est ce qui rend l'exercice instructif : le premier lève une erreur bien visible sur un moteur et se corrige en deux minutes, le second ne lève rien du tout sur l'autre et peut vivre des mois dans un rapport.

Théorie : `Theorie_01_SQL.md`, section 6.

E2. Elle filtre sur un agrégat dans le `WHERE`. À l'étape du `WHERE`, aucun `SUM` n'a encore été calculé, puisque le regroupement vient après : la condition est donc inévaluable, et le moteur la refuse. C'est le rôle exact du `HAVING`, qui s'exécute après le `GROUP BY`.

```sql
SELECT s.EnglishProductSubcategoryName, SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimProduct            AS p ON p.ProductKey = f.ProductKey
JOIN dbo.DimProductSubcategory AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
GROUP BY s.EnglishProductSubcategoryName
HAVING SUM(f.SalesAmount) > 100000
ORDER BY ventes DESC;
```

Si elle voulait au contraire ne compter que les ventes postérieures à 2012, la condition changerait de nature et donc de place. Elle porterait sur une ligne, pas sur un groupe, et devrait descendre dans le `WHERE`, avec la jointure vers `DimDate` que cela suppose. Les deux conditions peuvent d'ailleurs coexister dans la même requête, l'une filtrant les ventes avant le regroupement, l'autre les sous-catégories après. Le partage ne se décide pas au cas par cas, il se déduit de la question « cette condition parle-t-elle d'une ligne ou d'un paquet de lignes ».

Théorie : `Theorie_01_SQL.md`, section 6.

E3. Non, il n'a pas le droit de conclure. La colonne `ProductSubcategoryKey` de `DimProduct` contient des `NULL`, puisque certains produits n'ont pas de sous-catégorie. Or `x NOT IN (liste)` se réécrit `x <> v1 AND x <> v2 AND …` et l'une de ces comparaisons est `x <> NULL`, qui vaut inconnu. Une conjonction dont un membre est inconnu ne peut jamais valoir vrai : elle vaut inconnu ou faux. La condition n'est donc vraie pour aucune ligne, quelles que soient les données, et le résultat vide ne dit strictement rien.

© 2026 iAlexMG. Tous droits réservés.

La manipulation qui le prouve tient en deux lignes : on ajoute à la main une valeur qui n'existe certainement pas dans `DimProduct`, et on constate qu'elle ne ressort pas davantage.

```sql
SELECT -1 AS cle_bidon
WHERE -1 NOT IN (SELECT ProductSubcategoryKey FROM dbo.DimProduct);   -- toujours vide
```

Une requête qui ne peut rien renvoyer est pire qu'une requête qui échoue, parce qu'elle renvoie une réponse, et que cette réponse est plausible.

```sql
-- Correction 1 : NOT EXISTS, insensible aux NULL
SELECT s.EnglishProductSubcategoryName
FROM dbo.DimProductSubcategory AS s
WHERE NOT EXISTS (
    SELECT 1 FROM dbo.DimProduct AS p
    WHERE p.ProductSubcategoryKey = s.ProductSubcategoryKey
);

-- Correction 2 : garder le NOT IN, mais purger les NULL de la liste
SELECT EnglishProductSubcategoryName
FROM dbo.DimProductSubcategory
WHERE ProductSubcategoryKey NOT IN (
    SELECT ProductSubcategoryKey FROM dbo.DimProduct
    WHERE ProductSubcategoryKey IS NOT NULL
);
```

La première est celle que je recommande, parce qu'elle est correcte par construction et non parce qu'on a pensé à un filtre. La seconde marche, mais elle repose sur la vigilance de celui qui l'écrit, et cette vigilance sera absente le jour où quelqu'un modifiera la sous-requête.

Théorie : `Theorie_01_SQL.md`, section 8.

E4. La requête cherche au mauvais endroit. `Color` ne contient aucun `NULL` : les produits sans couleur y portent la valeur « NA », une vraie chaîne de deux lettres, que `IS NULL` ne voit pas. Le zéro est donc la bonne réponse à la question écrite, mais pas à la question voulue. C'est le pire des cas : aucune erreur, un chiffre plausible, une conclusion fausse.

Une seule requête l'aurait montré, celle qu'on lance devant toute colonne inconnue :

```sql
SELECT Color, COUNT(*) AS nb
FROM dbo.DimProduct
GROUP BY Color
ORDER BY nb DESC;
```

« NA » y apparaît en tête, avec 254 lignes. La correction ramène la sentinelle à un `NULL`, ce qui couvre aussi un vrai `NULL` s'il en apparaissait un jour :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT COUNT(*) AS sans_couleur
FROM dbo.DimProduct
WHERE NULLIF(Color, 'NA') IS NULL;    -- 254
```

La nuance : 254 lignes ne font pas 254 produits dont on ignore la couleur. D'abord, `DimProduct` garde plusieurs versions d'un même produit (Cours 00), donc on compte des lignes, pas des produits. Ensuite, « NA » regroupe deux réalités : 56 lignes de produits du catalogue pour lesquels la couleur n'a pas de sens, une fourche ou une pompe, et 198 lignes de pièces détachées, sans prix ni sous-catégorie, qui ne se vendent pas seules. Le rapport honnête dit « la couleur ne s'applique pas à 254 lignes du catalogue, dont 198 pièces détachées », et non « la couleur est inconnue pour 254 produits ».

Théorie : `Theorie_01_SQL.md`, section 9.

E5. Les 307 produits sans taille. Leur `Size` vaut `NULL`, et `NULL <> 'M'` vaut inconnu : le `WHERE` les écarte du second compte, sans erreur. Ils ne sont pourtant pas de taille M. La somme tombe juste avec `IS DISTINCT FROM`, qui compte l'absence comme une valeur :

```sql
SELECT COUNT(*) FROM dbo.DimProduct WHERE Size IS DISTINCT FROM 'M';    -- 592
```

14 et 592 font les 606 lignes. Un contrôle de ce genre, la somme des parties contre le tout, est le réflexe qui attrape ce piège : tant qu'on ne l'additionne pas, 285 a l'air d'une réponse.

Théorie : `Theorie_01_SQL.md`, section 8.

Questions à développer

A. Présenté comme une liste, l'ordre d'exécution est une chose à mémoriser puis à oublier. Présenté comme un outil, c'est une grille qui répond à une famille entière de questions avec une seule opération mentale : à cette étape, cette colonne existe-t-elle.

Trois erreurs qui n'ont l'air d'avoir aucun rapport. Un alias créé dans le `SELECT` et refusé dans le `WHERE` : l'alias naît à l'étape 5, le `WHERE` s'exécute à l'étape 2, donc non, il n'existe pas encore. Un `WHERE total > 100` refusé sur une somme : la somme naît à l'étape 3, le `WHERE` est en amont, donc non, elle n'existe pas encore, et il faut le `HAVING` qui vient après. Un `LEFT JOIN` qui se comporte comme un `INNER JOIN` : les `NULL` de la jointure naissent à l'étape 1, le `WHERE` les rencontre à l'étape 2 et les rejette, donc oui la colonne existe, mais elle vaut `NULL` et la condition la tue. Trois messages d'erreur différents, un seul raisonnement.

Ce que la grille n'explique pas, c'est tout ce qui relève du plan d'exécution réel. L'ordre décrit la sémantique, pas la mécanique : le moteur est libre de faire les choses dans un autre ordre tant que le résultat est le même, et c'est précisément ce qu'il fait quand il pousse un filtre à l'intérieur d'une jointure. La grille prédit ce que la requête va renvoyer, elle ne prédit ni sa vitesse ni son plan. Ce sera le Cours 10.

Théorie : `Theorie_01_SQL.md`, section 6.

B. Les deux versions existent parce qu'elles optimisent des choses opposées, et qu'aucun modèle ne peut optimiser les deux à la fois.

© 2026 iAlexMG. Tous droits réservés.

Le modèle transactionnel normalisé optimise l'écriture et la cohérence. Chaque information n'est stockée qu'à un seul endroit, donc la modifier est une opération unique, rapide, et qui ne peut pas laisser deux versions contradictoires derrière elle. Le prix payé est la lecture : reconstituer une vue d'ensemble demande de rassembler des morceaux éparpillés dans de nombreuses tables.

Le modèle entrepôt dénormalisé optimise la lecture analytique. Le contexte d'une vente est déjà rassemblé autour d'elle, les libellés sont résolus, les dates sont découpées. Le prix payé est la redondance, qui rendrait toute mise à jour hasardeuse, et le décalage, puisque cet arrangement est reconstruit périodiquement plutôt que tenu à jour en continu.

L'exemple dans les deux sens. Changer le nom d'un pays est trivial sur le transactionnel, une seule ligne dans le référentiel, et pénible dans l'entrepôt, où le libellé a été recopié dans des millions de lignes de dimension. Inversement, calculer les ventes par pays et par mois est immédiat sur l'entrepôt, deux jointures, et demande sur le transactionnel de joindre les lignes de commande aux en-têtes, puis au territoire, puis au référentiel des pays, en découpant soi-même la date. Ce n'est pas qu'un modèle soit meilleur, c'est que chacun a placé sa difficulté là où son usage ne la rencontre pas.

Théorie : `Theorie_01_SQL.md`, section 3.

C. Ce que `NULL` apporte est irremplaçable : il distingue « je ne sais pas » de « zéro » et de « vide ». Un produit dont le prix est `NULL` n'est pas un produit gratuit, et une date de fin `NULL` ne veut pas dire que le contrat s'est terminé en l'an zéro. Sans `NULL`, il faudrait inventer des valeurs sentinelles, un prix de moins un, une date au 1er janvier 1900, et ces sentinelles finissent toujours par entrer dans un calcul et par le fausser, sans qu'aucune règle du langage ne s'y oppose. `NULL`, au moins, est reconnu par le moteur, qui l'écarte des agrégats et refuse de le comparer.

Notre base montre les deux visages de la sentinelle. Le « NA » de `Color` est lisible dans un rapport, mais invisible pour `IS NULL`. Côté transactionnel, le prix à 0 des 200 pièces détachées fait tomber la moyenne des prix de 727 $ à 439 $, sans qu'aucune erreur ne le signale.

Ce qu'il coûte est réel aussi. Il introduit une logique à trois valeurs que presque personne n'applique correctement de tête, il fait taire silencieusement des conditions entières comme dans `NOT IN`, et il rend deux requêtes en apparence équivalentes différentes dans les faits. La plupart des bogues qu'il cause ne lèvent aucune erreur.

La règle que j'adopterais en concevant une table : `NOT NULL` par défaut sur toute colonne, et le `NULL` autorisé seulement quand l'absence de valeur a un sens métier qu'on sait énoncer en une phrase. Si personne dans l'équipe ne sait dire ce que veut dire un `NULL` dans cette colonne-là, c'est qu'il ne doit pas y en avoir. Et quand on l'autorise, on le documente, parce qu'une colonne nullable dont personne ne connaît la signification finira par être remplie de nulls qui veulent dire trois choses différentes.

Théorie : `Theorie_01_SQL.md`, sections 8 et 9.

Mises en situation

M1. Librairie Rive-Gauche.

Trois mécanismes distincts font perdre des livres, et il faut les voir séparément parce que chacun appelle une correction différente.

Le premier est la jointure. Un `JOIN` vers la table des rayons est une jointure interne : tout livre dont le rayon est vide, ou dont le rayon ne figure pas dans le référentiel, disparaît purement et simplement du rapport. C'est le piège du Cours 01, et c'est probablement le plus gros contributeur à l'écart.

© 2026 iAlexMG. Tous droits réservés.

Le deuxième est l'agrégat sur des `NULL`. Une quantité à `NULL` n'est pas comptée dans un `SUM`, ce qui est le comportement correct de SQL mais pas le comportement voulu ici : un livre dont la quantité est inconnue est un livre qui existe, et son absence du total est une décision qui n'a jamais été prise consciemment.

Le troisième est la différence entre `COUNT(*)` et `COUNT(colonne)`, si le rapport compte des titres. Compter une colonne qui peut être nulle donne un nombre de titres inférieur au nombre de lignes, et l'écart passe inaperçu tant que personne ne le cherche.

```sql
SELECT COALESCE(r.nom, 'Rayon non renseigné') AS rayon,
       SUM(COALESCE(l.quantite, 0))           AS exemplaires,
       COUNT(*)                               AS titres,
       COUNT(*) FILTER (WHERE l.quantite IS NULL) AS titres_sans_quantite
FROM livre AS l
LEFT JOIN rayon AS r ON r.id = l.rayon_id
GROUP BY 1
ORDER BY 1;
```

`GROUP BY 1` désigne la première colonne du `SELECT` (Cours 02, section 7), `ORDER BY 1` fait de même pour le tri, et `FILTER` restreint un agrégat à certaines lignes (Cours 06). Le `LEFT JOIN` et le `COALESCE` sur le rayon règlent le premier mécanisme. Le `COALESCE` sur la quantité règle le deuxième. Le `COUNT(*)` plutôt qu'un `COUNT(colonne)` règle le troisième. Et la colonne `titres_sans_quantite` ne corrige rien du tout : elle rend visible ce qu'on vient de masquer, ce qui est le seul moyen honnête d'utiliser un `COALESCE`.

Sur la question de fond, il faut faire les deux, mais pas dans le même temps ni pour la même raison. La requête se corrige tout de suite, parce que le rapport de ce soir doit être juste. Les données se corrigent ensuite, parce qu'une quantité à `NULL` et un rayon vide sont des informations manquantes qu'aucune requête ne peut inventer : tant qu'elles sont là, chaque nouvelle requête écrite par quelqu'un d'autre retombera dans le même trou. Ce qui décide, c'est la question de savoir si le `NULL` a un sens. Si un rayon vide veut dire « en attente de classement », alors c'est une donnée légitime et c'est aux requêtes de la traiter. Si ça ne veut rien dire, c'est un défaut de saisie, et le vrai correctif est une contrainte `NOT NULL` avec une valeur par défaut explicite.

Théorie : `Theorie_01_SQL.md`, sections 7, 8 et 9.

M2. Commission scolaire des Deux-Rives.

Les distinctions indispensables sont au nombre de trois, et elles se justifient toutes par une conséquence pratique, jamais par la définition.

Le langage contre le moteur, d'abord. C'est la seule confusion qui mène directement à de mauvaises décisions, et c'est donc celle sur laquelle il faut insister. Un exemple vaut mieux qu'une explication : une équipe qui croit qu'apprendre Oracle et apprendre SQL sont la même chose évaluera le coût d'une migration vers un autre moteur comme s'il fallait tout réécrire, alors que l'essentiel du code est portable ; ou, symétriquement, elle sous-estimera le travail en croyant que « c'est du SQL, donc ça marchera pareil », et découvrira les extensions propriétaires le jour du basculement. Les deux erreurs coûtent cher et viennent du même mot mal placé.

Le contenu contre le contenant, ensuite : la base de données est ce qui est stocké, le serveur est la machine qui l'héberge. La conséquence pratique est qu'on ne sauvegarde pas la même chose, et qu'une phrase comme « le serveur est plein » n'appelle pas la même action que « la base est pleine ».

© 2026 iAlexMG. Tous droits réservés.

Le troisième, la distinction entre données et information, est du pinaillage dans ce contexte : elle n'a pas de conséquence sur ce que l'équipe fait au quotidien, et l'imposer ferait passer tout le reste pour de la pédanterie.

Sur la manière, la seule qui fonctionne est de ne jamais corriger quelqu'un en réunion sur un mot. La page pour les nouveaux arrivants est le bon véhicule, précisément parce qu'elle ne s'adresse à personne en particulier. On l'écrit comme une aide, pas comme un rappel à l'ordre, et on la justifie par des exemples de décisions qui ont mal tourné ailleurs plutôt que par la norme. Une clarification de vocabulaire qui commence par « en réalité, ce n'est pas ça » est refusée avant d'être lue ; celle qui commence par « voici pourquoi la distinction compte pour nous » est adoptée.

Théorie : `Theorie_01_SQL.md`, section 1.

M3. `(avancé)` Distribution alimentaire Beauharnois.

Ce qui est juste dans son raisonnement : les données de la base de production sont effectivement plus fraîches, et le résultat de sa requête est effectivement le même, au décalage d'alimentation près. Il ne faut pas balayer ces deux points, parce qu'ils sont vrais et que les nier ferait perdre toute crédibilité au reste.

Ce qui est faux : « le résultat est le même » ne dit rien du coût de l'obtenir. Balayer trois ans d'historique de commandes sur un schéma normalisé demande plusieurs jointures et un parcours de volumes importants, sur une machine dont le travail est de servir des écritures courtes en continu. Et cela se répète toutes les cinq minutes, pour un tableau de bord que personne ne regarde plus de quelques fois par jour. Il est également faux, en général, que la base de production contienne l'historique dont il a besoin : les bases applicatives purgent, corrigent et écrasent, alors que l'entrepôt historise, ce qui veut dire que ses trois ans d'historique sont peut-être moins fiables qu'il ne le croit.

Ce que sa décision coûte à des gens absents de la conversation : les caissiers, les préparateurs de commandes et les clients. Une requête analytique lourde sur une base transactionnelle ne bloque personne, et c'est ce qui la rend sournoise : sur PostgreSQL, un `SELECT` ne pose qu'un verrou partagé sur la table, qui n'arrête que les changements de structure, et aucun verrou de ligne, ce que `pg_locks` confirme. Ce qu'elle prend, elle le prend ailleurs : les disques, la mémoire et surtout le cache, où trois ans de commandes chassent les pages chaudes dont l'application se sert à chaque seconde. Personne ne fera le lien entre un tableau de bord et une caisse qui rame, et c'est exactement ce qui rend ce genre de décision durablement nuisible : le coût est réel, il est payé par d'autres, et il n'est jamais attribué à sa cause.

À quelles conditions il aurait raison : si le besoin était vraiment opérationnel, par exemple suivre les commandes de la journée pour affecter des camions cet après-midi ; si le volume interrogé était petit et borné, quelques heures et non trois ans ; s'il se branchait sur une réplique en lecture plutôt que sur la base de production elle-même ; et si l'entrepôt ne contenait pas la donnée nécessaire, auquel cas le bon correctif serait de l'ajouter à l'alimentation.

La question qu'il aurait dû se poser en premier n'est pas « où sont les données les plus fraîches » mais « à quelle vitesse cette information doit-elle changer pour que la décision qu'elle sert soit la bonne ». C'est celle-là parce qu'elle est la seule qui relie le choix technique à un usage réel. La fraîcheur n'est pas une qualité en soi, c'est une exigence dérivée : un tableau de bord hebdomadaire rafraîchi toutes les cinq minutes ne prend pas de meilleures décisions, il consomme seulement davantage. Poser la question dans ce sens transforme un débat d'opinions en un calcul, et elle donne souvent une réponse que l'analyste trouve lui-même.

Théorie : `Theorie_01_SQL.md`, sections 3 et 5.

© 2026 iAlexMG. Tous droits réservés.